```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Http;
using System.Text.Json;
using System.Threading.Tasks;
using System.IO;
using BambooEnterprise.Models;
using BambooEnterprise.Data;
using BambooEnterprise.Testing;
using System.Globalization;

namespace BambooEnterprise
{
    class Program
    {
        static List<BambooSpecies> SpeciesDb = new();
        static List<InventoryBatch> InventoryDb = new();
        static List<PerformanceLog> GrowthDb = new();
        static List<OperationalExpense> ExpenseDb = new();
        private static readonly HttpClient _httpClient = new();
        private static System.Timers.Timer _autoSaveTimer;

        static void Main(string[] args)
        {
            try
            {
                SetupAutoSave();
                string invPath = Path.Combine
                  (AppDomain.CurrentDomain.BaseDirectory, "Data",
                  "inventory.json");
                if (File.Exists(invPath))
                {
                    Console.WriteLine("=== SYSTEM STARTUP ===");
                    Console.Write("Database detected. Load existing data?
                      (Y/N): ");
                    if (Console.ReadLine()?.ToUpper() == "Y")
                    {
                        LoadAllData();
                        Console.WriteLine("Data loaded successfully.");
                    }
                    else
                    {
                        SeedSampleData();
                        Console.WriteLine("System initialized with Sample
                          Data.");
                    }
                }
                else
```

```
                {
                    SeedSampleData();
                    Console.WriteLine("No database found. Initialized with Sample Data.");
                }
                Console.WriteLine("Press any key to enter Main Menu...");
                Console.ReadKey();
                MainMenu();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Startup Error: {ex.Message}");
                Console.ReadKey();
            }
        }

        private static void SetupAutoSave()
        {
            _autoSaveTimer = new System.Timers.Timer(300000);
            _autoSaveTimer.Elapsed += (sender, e) => PerformAutoSave();
            _autoSaveTimer.AutoReset = true;
            _autoSaveTimer.Enabled = true;
        }

        private static void PerformAutoSave()
        {
            PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
            PersistenceManager.SaveToJson(SpeciesDb, "species.json");
            PersistenceManager.SaveToJson(GrowthDb, "logs.json");
            PersistenceManager.SaveToJson(ExpenseDb, "expenses.json");
        }

        static void MainMenu()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("=== BAMBOO INFORMATION MANAGEMENT SYSTEM ===");
                ShowActiveAlerts();
                Console.WriteLine("\n1. Manage Biological Profiles (DNA)");
                Console.WriteLine("2. Track Inventory & Acquisition");
                Console.WriteLine("3. Log Growth & Harvests");
                Console.WriteLine("4. Operational Expenses (OpEx)");
                Console.WriteLine("5. Financial Reports (P&L)");
                Console.WriteLine("6. Database Management (Backup/Restore)");
                Console.WriteLine("Q. Quit Application");
```

```
                Console.Write("\nSelect: ");
                string choice = Console.ReadLine()?.ToUpper();
                if (choice == "Q") return;

                try { HandleChoice(choice); }
                catch (Exception ex) when (ex.Message == "MainMenu") { continue; }
                catch (Exception ex)
                {
                    Console.WriteLine($"\nApplication Error: {ex.Message}");
                    Console.WriteLine("Press any key to return to menu...");
                    Console.ReadKey();
                }
            }
        }

        static void HandleChoice(string choice)
        {
            switch (choice)
            {
                case "1": ManageSpeciesFlow(); break;
                case "2": ManageInventoryFlow(); break;
                case "3": ManageGrowthFlow(); break;
                case "4": ManageExpenseFlow(); break;
                case "5":
                    PersistenceManager.GeneratePLReport(InventoryDb, GrowthDb, ExpenseDb);
                    PersistenceManager.ExportToExcel(InventoryDb, "Inventory_Report");
                    Console.WriteLine("\nReports generated. Press any key...");
                    Console.ReadKey(); break;
                case "6":
                    Console.Clear();
                    Console.WriteLine("--- 6. DATABASE MANAGEMENT ---");
                    Console.WriteLine("1. Re-Seed Sample Data | 2. Restore from Backup | B. Back");
                    string dbChoice = Console.ReadLine()?.ToUpper();
                    if (dbChoice == "1") { SeedSampleData(); Console.WriteLine("Data Re-Seeded."); Console.ReadKey(); }
                    else if (dbChoice == "2") RestoreFlow();
                    break;
                default:
                    Console.WriteLine("\n!!! Invalid selection. Press any key...");
```

```
                    Console.ReadKey(); break;
            }
        }

        // --- CORE MODULES ---

        static void ManageSpeciesFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- BIOLOGICAL PROFILES ---");
                Console.WriteLine("S. Search | V. View All | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;

                IEnumerable<BambooSpecies> displayList = SpeciesDb;
                if (cmd == "S")
                {
                    Console.Write("Search term: ");
                    string filter = Console.ReadLine()?.ToLower();
                    displayList = SpeciesDb.Where(s => ($"{s.Genus} {s.Species} {s.Cultivar}").ToLower().Contains(filter));
                }

                int globalCounter = 1;
                foreach (var s in displayList)
                {
                    if ((globalCounter - 1) % 10 == 0)
                        Console.WriteLine("\n{0,-4} {1,-12} {2,-42} {3,-6} {4,-6} {5}", "Num", "UID", "Scientific Name \"Cultivar\"", "Hardy", "MaxHt", "Research Notes");

                    string name = $"{s.Genus} {s.Species} \"{s.Cultivar}\"".Trim();
                    Console.WriteLine("{0,-4} {1,-12} {2,-42} {3,-6} {4,-6} {5}", globalCounter, s.PlantUID, name, s.HardinessLow + "C", s.HeightHigh + "m", s.SverigeSpecifikt);
                    globalCounter++;
                }
                Console.WriteLine("\nPress any key..."); Console.ReadKey();
                if (cmd != "S" && cmd != "V") break;
            }
        }

```

```
        static void ManageInventoryFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 2. TRACK INVENTORY ---");
                Console.WriteLine("1. View All | 2. Summary View | 3. Add | 4. Update | 5. Delete | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;

                if (cmd == "1") ViewInventory();
                else if (cmd == "2") ViewInventorySummary();
                else if (cmd == "3") AddInventoryFlow().GetAwaiter().GetResult();
                else if (cmd == "4") UpdateInventoryFlow().GetAwaiter().GetResult();
                else if (cmd == "5") DeleteInventoryFlow();
            }
        }

        static void ViewInventorySummary()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- INVENTORY SUMMARY VIEW ---");
                Console.WriteLine("1. Group by Nursery | 2. Group by Date | 3. Group by Species | B. Back");
                string mode = Console.ReadLine()?.ToUpper();
                if (mode == "B") return;

                var enrichedInventory = InventoryDb.Join(SpeciesDb,
                    inv => inv.PlantUID,
                    spec => spec.PlantUID,
                    (inv, spec) => new {
                        inv.BatchID,
                        inv.Nursery,
                        inv.PurchaseDate,
                        inv.Quantity,
                        inv.TotalCostSek,
                        inv.UnitCost,
                        inv.Curr,
                        FullName = $"{spec.Genus} {spec.Species} {(string.IsNullOrEmpty(spec.Cultivar) ? "" : $"'{spec.Cultivar}'")}"
                    }).ToList();

                string exportContent = $"INVENTORY SUMMARY -
```

```
                    {DateTime.Now:yyyy-MM-dd HH:mm}\n" + new string('=', 60) + "\n";
                bool valid = true;

                var groups = mode switch
                {
                    "1" => enrichedInventory.GroupBy(b => b.Nursery).OrderBy(x => x.Key).Select(g => new { Header = $"NURSERY: {g.Key.ToUpper()}", Items = g }),
                    "2" => enrichedInventory.GroupBy(b => b.PurchaseDate.ToString("yyyy-MM")).OrderBy(x => x.Key).Select(g => new { Header = $"PERIOD: {g.Key}", Items = g }),
                    "3" => enrichedInventory.GroupBy(b => b.FullName).OrderBy(x => x.Key).Select(g => new { Header = $"SPECIES: {g.Key}", Items = g }),
                    _ => null
                };

                if (groups != null)
                {
                    foreach (var g in groups)
                    {
                        string headerLine = $"\n{g.Header} | Total: {g.Items.Sum(b => b.TotalCostSek):N2} SEK";
                        Console.WriteLine(headerLine);
                        exportContent += headerLine + "\n";
                        foreach (var b in g.Items)
                        {
                            string line = $" [{b.BatchID}] {b.FullName} - {b.Quantity} units";
                            Console.WriteLine(line);
                            exportContent += line + "\n";
                        }
                    }
                }
                else { valid = false; }

                if (valid)
                {
                    Console.WriteLine("\nExport summary? (1. Text | 2. Excel | 3. PDF | B. Return)");
                    string choice = Console.ReadLine()?.ToUpper();
                    if (choice == "B") return;

                    if (choice == "1") PersistenceManager.ExportSummary(exportContent, "Inventory_Summary");
                    else if (choice == "2")
```

```
                        PersistenceManager.ExportToExcel(enrichedInventory, "Inventory_Full_Export");
                    else if (choice == "3")
                        PersistenceManager.GeneratePLReport(InventoryDb, GrowthDb, ExpenseDb);

                    Console.WriteLine("\nPress any key to continue...");
                    Console.ReadKey();
                }
            }
        }

        static void ViewInventory()
        {
            Console.Clear();
            Console.WriteLine("{0,-18} | {1,-10} | {2,-12} | {3,-5} | {4,-5} | {5,-10} | {6,-4} | {7,-10} | {8,-10}",
                "BATCH ID", "DATE", "NURSERY", "POT", "QTY", "COST", "CURR", "UNIT SEK", "TOTAL SEK");
            Console.WriteLine(new string('-', 125));

            foreach (var b in InventoryDb)
            {
                double unitSek = b.UnitCost * b.ExchangeRateToSek;
                Console.WriteLine("{0,-18} | {1,-10:yyyy-MM-dd} | {2,-12} | {3,-5} | {4,-5} | {5,-10:N2} | {6,-4} | {7,-10:N2} | {8,-10:N2}",
                    b.BatchID, b.PurchaseDate, b.Nursery, b.PotVolume, b.Quantity, b.UnitCost, b.Curr, unitSek, b.TotalCostSek);
            }

            Console.WriteLine(new string('-', 125));
            Console.WriteLine("{0,103} | Total: {1:N2} SEK", "", InventoryDb.Sum(x => x.TotalCostSek));
            Console.WriteLine("\nPress any key..."); Console.ReadKey();
        }

        static async Task AddInventoryFlow()
        {
            try
            {
                while (true)
                {
                    Console.Clear();
                    Console.WriteLine("--- ADD NEW INVENTORY ---");
                    DateTime date = SafeRequestDate("Purchase Date (YYYYMMDD or YYYY-MM-DD)");
                    string nursery = RequestInput("Nursery Name", true);
```

```
                    string uid = SelectPlantUID();
                    double pot = SafeRequestDouble("Pot Volume (L)");
                    int qty = SafeRequestInt("Quantity");
                    double cost = SafeRequestDouble("Unit Cost");

                    Console.WriteLine("\nREVIEW ENTRY:");
                    Console.WriteLine($"Date: {date:yyyy-MM-dd} | Nursery: {nursery} | UID: {uid}");
                    Console.WriteLine($"Qty: {qty} | Cost: {cost} | Pot: {pot}L");

                    Console.Write("\nSave record? (Y/N/R to Restart): ");
                    string confirm = Console.ReadLine()?.ToUpper();
                    if (confirm == "R") continue;
                    if (confirm != "Y") return;

                    var batch = new InventoryBatch
                    {
                        BatchID = $"{date:yyyyMMdd}-{nursery.Substring(0, Math.Min(3, nursery.Length)).ToUpper()}-{InventoryDb.Count + 1:D3}",
                        PlantUID = uid,
                        PurchaseDate = date,
                        Nursery = nursery,
                        PotVolume = pot,
                        Quantity = qty,
                        UnitCost = cost,
                        ExchangeRateToSek = 1.0, // Default for manual entry
                        UpdateReason = "Initial Entry",
                        LastUpdatedDate = DateTime.Now
                    };

                    InventoryDb.Add(batch);
                    PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                    Console.WriteLine("Saved."); Console.ReadKey();
                    break;
                }
            }
            catch (Exception ex) when (ex.Message == "GoBack") { return; }
        }

        static async Task UpdateInventoryFlow()
        {
            string bid = SelectBatchID();
            if (bid == null) return;
            var b = InventoryDb.First(x => x.BatchID == bid);

```

```
            Console.Clear();
            Console.WriteLine("--- UPDATE RECORD ---");
            DisplayBatchDetails(b);
            Console.WriteLine("\nPress Enter to keep current value, or type new value.");

            // Cycling through fields
            string nurseryInput = RequestInput($"Nursery ({b.Nursery})", false);
            if (!string.IsNullOrEmpty(nurseryInput)) b.Nursery = nurseryInput;

            string potInput = RequestInput($"Pot Volume ({b.PotVolume})", false);
            if (double.TryParse(potInput, out double p)) b.PotVolume = p;

            string qtyInput = RequestInput($"Quantity ({b.Quantity})", false);
            if (int.TryParse(qtyInput, out int q)) b.Quantity = q;

            string costInput = RequestInput($"Unit Cost ({b.UnitCost})", false);
            if (double.TryParse(costInput, out double c)) b.UnitCost = c;

            string reason = RequestInput("Update Reason", true);

            Console.WriteLine("\nPROPOSED CHANGES:");
            DisplayBatchDetails(b);
            Console.Write("\nConfirm Update? (Y/N): ");
            if (Console.ReadLine()?.ToUpper() == "Y")
            {
                b.UpdateReason = reason;
                b.LastUpdatedDate = DateTime.Now;
                PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                Console.WriteLine("Update saved.");
            }
            else
            {
                Console.WriteLine("Update cancelled. Reloading original data...");
                LoadAllData();
            }
            Console.ReadKey();
        }

        static void DeleteInventoryFlow()
        {
            string bid = SelectBatchID();
```

```
            if (bid == null) return;
            var b = InventoryDb.First(x => x.BatchID == bid);

            Console.Clear();
            Console.WriteLine("--- DELETE CONFIRMATION ---");
            DisplayBatchDetails(b);

            Console.Write($"\nARE YOU SURE YOU WANT TO DELETE {bid}? (Y/N): ");
            if (Console.ReadLine()?.ToUpper() == "Y")
            {
                InventoryDb.Remove(b);
                PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                Console.WriteLine("Deleted.");
            }
            else { Console.WriteLine("Deletion cancelled."); }
            Console.ReadKey();
        }

        static void DisplayBatchDetails(InventoryBatch b)
        {
            Console.WriteLine(new string('-', 40));
            Console.WriteLine($"ID:       {b.BatchID}");
            Console.WriteLine($"UID:      {b.PlantUID}");
            Console.WriteLine($"Date:     {b.PurchaseDate:yyyy-MM-dd}");
            Console.WriteLine($"Nursery:  {b.Nursery}");
            Console.WriteLine($"Pot:      {b.PotVolume}L");
            Console.WriteLine($"Quantity: {b.Quantity}");
            Console.WriteLine($"Cost:     {b.UnitCost} {b.Curr}");
            Console.WriteLine($"Total:    {b.TotalCostSek:N2} SEK");
            Console.WriteLine(new string('-', 40));
        }

        static void ManageGrowthFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 3. GROWTH ---");
                Console.WriteLine("1. View | 2. Add | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                if (cmd == "1")
                {
                    foreach (var l in GrowthDb) Console.WriteLine($"{l.Date:yyyy-MM} | Batch {l.BatchID} | {l.MaxHeightCurrent}m");
                    Console.ReadKey();
```

```
                }
                else if (cmd == "2")
                {
                    string bid = RequestInput("Batch ID", true);
                    GrowthDb.Add(new PerformanceLog { LogID = Guid.NewGuid
                      ().ToString().Substring(0, 8), BatchID = bid,
                      Date = DateTime.Now, MaxHeightCurrent =
                      SafeRequestDouble("Height") });
                    PersistenceManager.SaveToJson(GrowthDb, "logs.json");
                    Console.WriteLine("Growth saved."); Console.ReadKey();
                }
            }
        }

        static void ManageExpenseFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 4. EXPENSES ---");
                Console.WriteLine("1. View | 2. Add | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                if (cmd == "1")
                {
                    foreach (var e in ExpenseDb) Console.WriteLine
                      ($"{e.Date:yyyy-MM} | {e.BatchID} |
                      {e.MaintenanceCost} SEK");
                    Console.ReadKey();
                }
                else if (cmd == "2")
                {
                    string bid = RequestInput("Batch ID (or 'Global')",
                      true);
                    ExpenseDb.Add(new OperationalExpense { ExpenseID =
                      Guid.NewGuid().ToString().Substring(0, 8),
                      BatchID = bid, Date = DateTime.Now, MaintenanceCost
                      = SafeRequestDouble("Cost") });
                    PersistenceManager.SaveToJson(ExpenseDb,
                      "expenses.json");
                    Console.WriteLine("Expense saved."); Console.ReadKey
                      ();
                }
            }
        }

        // --- HELPERS & FLOW CONTROL ---

        private static string SelectPlantUID()
```

```
        {
            for (int i = 0; i < SpeciesDb.Count; i++) Console.WriteLine($"{i + 1}. {SpeciesDb[i].PlantUID}");
            int idx = SafeRequestInt("Select #");
            return SpeciesDb[idx - 1].PlantUID;
        }

        private static string SelectBatchID()
        {
            for (int i = 0; i < InventoryDb.Count; i++) Console.WriteLine($"{i + 1}. {InventoryDb[i].BatchID}");
            string input = Console.ReadLine();
            if (input?.ToUpper() == "B") return null;
            if (int.TryParse(input, out int idx) && idx > 0 && idx <= InventoryDb.Count) return InventoryDb[idx - 1].BatchID;
            Console.WriteLine("Invalid selection."); Console.ReadKey();
            return null;
        }

        static void RestoreFlow()
        {
            var backups = PersistenceManager.GetAvailableBackups();
            if (!backups.Any()) { Console.WriteLine("No backups."); Console.ReadKey(); return; }

            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- RESTORE FROM BACKUP ---");
                for (int i = 0; i < backups.Count; i++) Console.WriteLine($"{i + 1}. {backups[i].Name}");
                Console.WriteLine("B. Back");

                string input = RequestInput("Select # to restore", true);
                if (input.ToUpper() == "B") return;

                if (int.TryParse(input, out int idx) && idx > 0 && idx <= backups.Count)
                {
                    var selected = backups[idx - 1];
                    // Peek into backup
                    var backupContent = JsonSerializer.Deserialize<List<InventoryBatch>>(File.ReadAllText(selected.FullName));
                    Console.WriteLine($"\nBackup contains {backupContent?.Count ?? 0} records.");
                    Console.Write("Overwrite current database with this backup? (Y/N): ");

```

```
                    if (Console.ReadLine()?.ToUpper() == "Y")
                    {
                        PersistenceManager.RestoreBackup (selected.FullName, "inventory.json");
                        LoadAllData();
                        Console.WriteLine("Database restored."); Console.ReadKey();
                        return;
                    }
                    else { Console.WriteLine("Restore aborted."); Console.ReadKey(); return; }
                }
                else { Console.WriteLine("Invalid input. Please enter a number or 'B'."); Console.ReadKey(); }
            }
        }

        static string RequestInput(string p, bool r)
        {
            Console.Write($"{p}: ");
            string i = Console.ReadLine();
            if (i?.ToUpper() == "B") throw new Exception("GoBack");
            if (r && string.IsNullOrWhiteSpace(i)) { Console.WriteLine("Required field."); return RequestInput(p, r); }
            return i;
        }

        static double SafeRequestDouble(string p) { while (true) try { return double.Parse(RequestInput(p, true)); } catch { Console.WriteLine("Invalid number."); } }
        static int SafeRequestInt(string p) { while (true) try { return int.Parse(RequestInput(p, true)); } catch { Console.WriteLine("Invalid integer."); } }

        static DateTime SafeRequestDate(string p)
        {
            string[] formats = { "yyyy-MM-dd", "yyyyMMdd" };
            while (true)
            {
                string input = RequestInput(p, true);
                if (DateTime.TryParseExact(input, formats, CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime res)) return res;
                Console.WriteLine("Invalid date. Use YYYY-MM-DD or YYYYMMDD.");
            }
        }

        static void ShowActiveAlerts() { int m = DateTime.Now.Month; if (
```

```
            m >= 5 && m <= 7) Console.WriteLine("!!! ALERT: Peak Edible Shoot Harvest !!!"); }
        static void SeedSampleData() { SpeciesDb = SeedDataGenerator.GetResearchedSpecies(); InventoryDb = SeedDataGenerator.GetSampleInventoryBatches(); PerformAutoSave(); }
        static void LoadAllData() { SpeciesDb = PersistenceManager.LoadFromJson<BambooSpecies>("species.json"); InventoryDb = PersistenceManager.LoadFromJson<InventoryBatch>("inventory.json"); GrowthDb = PersistenceManager.LoadFromJson<PerformanceLog>("logs.json"); ExpenseDb = PersistenceManager.LoadFromJson<OperationalExpense>("expenses.json"); }
    }
}
```

```
using BambooEnterprise.Models;
using DocumentFormat.OpenXml.Drawing;
using DocumentFormat.OpenXml.Math;
using DocumentFormat.OpenXml.Office2013.Word;
using DocumentFormat.OpenXml.Spreadsheet;
using DocumentFormat.OpenXml.Wordprocessing;
using iText.StyledXmlParser.Jsoup.Nodes;
using iText.StyledXmlParser.Jsoup.Select;
using Org.BouncyCastle.Ocsp;
using System;
using System.Collections.Generic;
using System.Diagnostics.Contracts;
using System.Runtime.InteropServices;
using static Org.BouncyCastle.Asn1.Cmp.Challenge;
namespace BambooEnterprise.Testing
{
    public static class SeedDataGenerator
    {
    // Hardcoded Sample Inventory Data Moved to the top for easy management
    // Defined once at the class level
    public static readonly List<InventoryBatch> SampleInventory = new
    List<InventoryBatch>
    {
        new InventoryBatch {
            BatchID = "20220611-KIM-001",
            PlantUID = "PHY-VIV-AUR",
            PurchaseDate = new DateTime(2022, 06, 11),
            Nursery = "Kimmei",
            PotVolume = 10,
            Quantity = 2,
            UnitCost = 45.00,
            Curr = Currency.EUR,
            ExchangeRateToSek = 11.20,
            UpdateReason = "System Seed Data",
            LastUpdatedDate = DateTime.Now
        },
        new InventoryBatch {
            BatchID = "20220611-KIM-002",
            PlantUID = "PHY-PAR-NOM",
            PurchaseDate = new DateTime(2022, 06, 11),
            Nursery = "Kimmei",
            PotVolume = 5,
            Quantity = 1,
            UnitCost = 35.00,
            Curr = Currency.EUR,
            ExchangeRateToSek = 11.20,
            UpdateReason = "System Seed Data",
            LastUpdatedDate = DateTime.Now
```

```
        }
    };
        public static List<InventoryBatch> GetSampleInventoryBatches() =>
        SampleInventory;
        public static List<BambooSpecies> GetResearchedSpecies()
        {
            var list = new List<BambooSpecies>();
            // Runners
            list.Add(CreateMono("PHY-VIV-HUA", "Phyllostachys", "vivax", "Huanwenzhu", 12, 10, -21, 6, "Green culm, yellow sulcus.Snöbrott risk."));
            list.Add(CreateMono("PHY-VIV-MCC", "Phyllostachys", "vivax", "McClure", 14, 12, -21, 6, "Classic green timber. Large leaves."));
            list.Add(CreateMono("PHY-VIV-AUR", "Phyllostachys", "vivax", "Aureocaulis", 12, 11, -21, 6, "Yellow culm, green stripes. High aesthetics."));
            list.Add(CreateMono("PHY-PAR-NOM", "Phyllostachys", "parvifolia", "", 10, 9, -25, 5, "Extremely hardy timber. Edible shoots."));
            list.Add(CreateMono("PHY-ATR-INC", "Phyllostachys","atrovaginata", "", 8, 7, -23, 5, "Air canals for wet soil. Scented."));
            list.Add(CreateMono("PHY-AUR-NOM", "Phyllostachys", "aureosulcata", "", 7, 4, -25, 5, "Green culm, yellow sulcus.Zig - zag bases."));
            list.Add(CreateMono("PHY-AUR-AUR", "Phyllostachys", "aureosulcata", "Aureocaulis", 7, 4, -25, 5, "Solid yellow. Very hardy."));
            list.Add(CreateMono("PHY-AUR-SPE", "Phyllostachys", "aureosulcata", "Spectabilis", 7, 4, -25, 5, "Yellow culm, green sulcus.Red shoots."));
            list.Add(CreateMono("PHY-AUR-ARG", "Phyllostachys", "aureosulcata", "Argus", 7, 4, -25, 5, "Green stripes on yellow."));
            list.Add(CreateMono("PHY-NIG-NOM", "Phyllostachys", "nigra", "", 5, 3, -18, 5, "Jet black culms. Needs shelter."));
            list.Add(CreateMono("PHY-NIG-HEN", "Phyllostachys", "nigra", "Henonis", 10, 8, -23, 6, "Giant grey timber. Very hardy."));
            list.Add(CreateMono("PHY-NIG-PUN", "Phyllostachys", "nigra", "Punctata", 7, 4, -20, 6, "Speckled black culms."));
            list.Add(CreateMono("PHY-BIS-NOM", "Phyllostachys", "bissetii", "", 7, 4, -26, 4, "Best Swedish windbreak."));
            list.Add(CreateMono("PHY-DEC-NOM", "Phyllostachys", "decora", "", 6, 4, -22, 5, "Drought tolerant runner."));
            list.Add(CreateMono("PHY-NUD-NOM", "Phyllostachys", "nuda", "", 6, 4, -26, 4, "Early purple / black nodes."));
            list.Add(CreateMono("SAS-KUR-NOM", "Sasa", "kurilensis", "",
```

```
                2, 1.5, -30, 6, "Northernmost species. Large leaves."));
            list.Add(CreateMono("BAS-QIN-NOM", "Bashania","qingchengshanensis", "", 4, 2, -20, 4, "Strong, stiff culms."));
            list.Add(CreateMono("SEM-FAS-NOM", "Semiarundinaria", "fastuosa", "", 8, 4, -20, 6, "Upright columnar habit."));
            // Clumpers
            list.Add(CreateSym("FAR-NIT-OBE", "Fargesia", "nitida x murieliae", "Obelisk", 4, -25, 5, "Tallest upright columnar hedge."));
            list.Add(CreateSym("FAR-MUR-RZB", "Fargesia", "murieliae", "Red Zebra", 3, -23, 5, "Dark red / green contrast stems."));
            list.Add(CreateSym("FAR-NIT-SCH", "Fargesia", "nitida x murieliae", "Schensbossen", 4, -26, 5, "Exceptional hardiness."));
            list.Add(CreateSym("FAR-SPP-RUF", "Fargesia", "sp.", "Rufa", 2.5, -23, 4, "Non-curling leaves. Early shoots."));
            list.Add(CreateSym("FAR-ROB-CAM", "Fargesia", "robusta","Campbell", 5, -20, 4, "Earliest shoots. White pattern."));
            list.Add(CreateSym("FAR-NIT-JUR", "Fargesia", "nitida", "Jürgen", 3.5, -26, 5, "Upright, dark culms."));
            list.Add(CreateSym("FAR-JIU-ONE", "Fargesia", "sp. Jiuzhaigou", "1", 3, -25, 5, "Red Panda.Cherry stems in sun."));
            list.Add(CreateSym("FAR-JIU-DPU", "Fargesia", "sp. Jiuzhaigou", "Deep Purple", 3, -25, 5, "Deepest purple culm color."));
            list.Add(CreateSym("FAR-DEM-GER", "Fargesia", "demissa", "Gerry", 3, -25, 5, "Almost black culms."));
            list.Add(CreateSym("FAR-DEN-XI2", "Fargesia", "denudata", "Xian 2", 4, -22, 5, "Cloud - like elegant habit."));
            list.Add(CreateSym("FAR-SPP-KR5", "Fargesia", "sp.", "KR5287", 4, -20, 6, "Rare collector blueish culms."));
return list;
        }
        private static Monopodial CreateMono(string uid, string g, string
        s, string c, double h, double d, double hard, int shoot, string
        note)
        => new Monopodial
        {
            PlantUID = uid,
            Genus = g,
            Species = s,
            Cultivar = c,
            Habit = GrowthHabit.Monopodial,
            HeightHigh =
        h,
```

```
            DiameterHigh = d,
            HardinessLow = hard,
            TypicalShootingMonth = shoot,
            SverigeSpecifikt = note,
            ShootEdibilityRank = 4,
            HeightLow = h * 0.7,
            DiameterLow = d * 0.6
        };
        private static Sympodial CreateSym(string uid, string g, string s,
        string c, double h, double hard, int shoot, string note)
        => new Sympodial
        {
            PlantUID = uid,
            Genus = g,
            Species = s,
            Cultivar = c,
            Habit = GrowthHabit.Sympodial,
            HeightHigh = h,
            HardinessLow = hard,
            TypicalShootingMonth = shoot,
            SverigeSpecifikt = note,
            ClumpSpacing = 1.0,
            HeightLow = h * 0.7
        };
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;

namespace BambooEnterprise.Models
{
    public enum GrowthHabit { Monopodial, Sympodial }
    public enum Currency { SEK, EUR, USD, GBP }

    [JsonDerivedType(typeof(Monopodial), typeDiscriminator: "monopodial")]
    [JsonDerivedType(typeof(Sympodial), typeDiscriminator: "sympodial")]
    public abstract class BambooSpecies
    {
        public string PlantUID { get; set; }
        public string Genus { get; set; }
        public string Species { get; set; }
        public string Cultivar { get; set; }
        public GrowthHabit Habit { get; set; }
        public double HeightLow { get; set; }
        public double HeightHigh { get; set; }
        public double DiameterLow { get; set; }
        public double DiameterHigh { get; set; }
        public double HardinessLow { get; set; }
        public int TypicalShootingMonth { get; set; }
        public string SverigeSpecifikt { get; set; }
        public int ShootEdibilityRank { get; set; }
        public int TimberQualityRank { get; set; }
    }

    public class Monopodial : BambooSpecies { public double RhizomeDepth { get; set; } }
    public class Sympodial : BambooSpecies { public double ClumpSpacing { get; set; } }

    public class InventoryBatch
    {
        public string BatchID { get; set; }
        public string PlantUID { get; set; }
        public DateTime PurchaseDate { get; set; }
        public string Nursery { get; set; }
        public double PotVolume { get; set; }
        public int Quantity { get; set; }
        public double UnitCost { get; set; }
        public Currency Curr { get; set; }
        public double ExchangeRateToSek { get; set; }
        public double TotalCostSek => (UnitCost * Quantity) * ExchangeRateToSek;

        // Audit Fields
```

```
        public string UpdateReason { get; set; }
        public DateTime LastUpdatedDate { get; set; }

        public bool IsPeakCulmAge(int currentYear) => (currentYear - PurchaseDate.Year) >= 4;
    }

    public class PerformanceLog
    {
        public string LogID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public double MaxHeightCurrent { get; set; }
        public double CulmRevenueSek { get; set; }
        public double ShootRevenueSek { get; set; }
    }

    public class OperationalExpense
    {
        public string ExpenseID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public double MaintenanceCost { get; set; }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.Json;
using BambooEnterprise.Models;
using ClosedXML.Excel;
using iText.IO.Font.Constants;
using iText.Kernel.Font;
using iText.Kernel.Pdf;
using iText.Layout;
using iText.Layout.Element;
using iText.Layout.Properties;

namespace BambooEnterprise.Data
{
    public static class PersistenceManager
    {
        private static readonly string Root = AppDomain.CurrentDomain.BaseDirectory;
        private static readonly string DataDir = Path.Combine(Root, "Data");
        private static readonly string BackupDir = Path.Combine(DataDir, "Backups");
        private static readonly string PdfDir = Path.Combine(Root, "Reports", "PDF");
        private static readonly string ExcelDir = Path.Combine(Root, "Reports", "Excel");
        private static readonly string SummaryDir = Path.Combine(Root, "Reports", "Summary");

        static PersistenceManager()
        {
            Directory.CreateDirectory(DataDir);
            Directory.CreateDirectory(BackupDir);
            Directory.CreateDirectory(PdfDir);
            Directory.CreateDirectory(ExcelDir);
            Directory.CreateDirectory(SummaryDir);
        }

        private static void CreateBackup(string fileName)
        {
            try
            {
                string source = Path.Combine(DataDir, fileName);
                if (File.Exists(source))
                {
                    string bName = $"{Path.GetFileNameWithoutExtension(fileName)}_{DateTime.Now:yyyyMMdd_HHmmss}.json";
```

```
                    string bPath = Path.Combine(BackupDir, bName);
                    File.Copy(source, bPath, true);
                    Console.WriteLine($"[BACKUP] Created: {bPath}");
                    var old = new DirectoryInfo(BackupDir).GetFiles
                      ("*.json").OrderByDescending(f =>
                      f.CreationTime).Skip(10);
                    foreach (var f in old) f.Delete();
                }
            }
            catch { }
        }

        public static void SaveToJson<T>(List<T> data, string fileName)
        {
            try
            {
                CreateBackup(fileName);
                string path = Path.Combine(DataDir, fileName);
                File.WriteAllText(path, JsonSerializer.Serialize(data,
                  new JsonSerializerOptions { WriteIndented = true }));
                Console.WriteLine($"[SAVE] Successful: {path}");
            }
            catch (Exception ex) { throw new Exception($"Save Failed:
              {ex.Message}"); }
        }

        public static List<T> LoadFromJson<T>(string fileName)
        {
            string path = Path.Combine(DataDir, fileName);
            if (!File.Exists(path)) return new List<T>();
            try
            {
                Console.WriteLine($"[LOAD] Reading: {path}");
                return JsonSerializer.Deserialize<List<T>>
                  (File.ReadAllText(path)) ?? new List<T>();
            }
            catch { return new List<T>(); }
        }

        public static List<FileInfo> GetAvailableBackups() => new
          DirectoryInfo(BackupDir).GetFiles("*.json").OrderByDescending(f
          => f.CreationTime).ToList();

        public static void RestoreBackup(string source, string target)
        {
            string targetPath = Path.Combine(DataDir, target);
            CreateBackup(target);
            File.Copy(source, targetPath, true);
            Console.WriteLine($"[RESTORE] Overwritten: {targetPath}");
```

```
        }

        public static void ExportSummary(string content, string name)
        {
            string path = Path.Combine(SummaryDir, $"{name}_{DateTime.Now:yyyyMMdd}.txt");
            File.WriteAllText(path, content);
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }

        public static void ExportToExcel<T>(List<T> data, string name)
        {
            string path = Path.Combine(ExcelDir, $"{name}.xlsx");
            using (var wb = new XLWorkbook()) { wb.Worksheets.Add("Report").Cell(1, 1).InsertTable(data); wb.SaveAs(path); }
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }

        public static void GeneratePLReport(List<InventoryBatch> batches, List<PerformanceLog> logs, List<OperationalExpense> expenses)
        {
            string path = Path.Combine(PdfDir, "Bamboo_PL_Report.pdf");
            using (var writer = new PdfWriter(path))
            using (var pdf = new PdfDocument(writer))
            using (var doc = new Document(pdf))
            {
                doc.Add(new Paragraph("Profit & Loss Report").SetFontSize(18));
                Table table = new Table(5).SetWidth(UnitValue.CreatePercentValue(100)); // 5 columns
                table.AddHeaderCell("Batch"); table.AddHeaderCell("Revenue"); table.AddHeaderCell("Costs"); table.AddHeaderCell("Net"); table.AddHeaderCell("Audit Reason (Date)");
                foreach (var b in batches)
                {
                    double rev = logs.Where(l => l.BatchID == b.BatchID).Sum(l => l.CulmRevenueSek + l.ShootRevenueSek);
                    double cost = expenses.Where(e => e.BatchID == b.BatchID || e.BatchID == "Global").Sum(e => e.MaintenanceCost) + b.TotalCostSek;
                    table.AddCell(b.BatchID); table.AddCell(rev.ToString("N2")); table.AddCell(cost.ToString("N2")); table.AddCell((rev - cost).ToString("N2"));
                    table.AddCell($"{b.UpdateReason} ({b.LastUpdatedDate:yyyy-MM-dd})");
                }
                doc.Add(table);
```

```
            }
            Console.WriteLine($"[PDF EXPORT] Saved: {path}");
        }
    }
}

```